EXERCICE 14 · CODE COMPLET

Le panier d'achat avec Redux Toolkit

Tous les fichiers utilisés, en entier, dans l'ordre de lecture.

1. **correction** (13 fichiers)

1 correction

Styles fournis (copiés tels quels, pas tapés en live) : src/styles.css (247 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : vite.config.js, .gitignore, eslint.config.js, public/vite.svg

package.json (gabarit Vite) : dépendances ajoutées : @reduxjs/toolkit ^2.11.2, react-redux ^9.2.0

FICHIER **correction/src/main.jsx** 4 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(<App />);
```

FICHIER **correction/src/App.jsx** 14 lignes

```jsx
import "./App.css";
import Home from "./pages/Home";
import { Provider } from "react-redux";
import { store } from "./store/store";

function App() {
  return (
    <Provider store={store}>
      <Home />
    </Provider>
  );
}

export default App;
```

FICHIER **correction/src/App.css** 16 lignes

```css
/* Import de la police Google Fonts "Inter" avec différentes épaisseurs */
@import url("https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap");

/* Reset global pour tous les éléments */
* {
  box-sizing: border-box; /* Inclut padding et border dans la largeur/hauteur totale */
  margin: 0; /* Supprime toutes les marges par défaut */
  padding: 0; /* Supprime tous les paddings par défaut */
}

body {
  font-family: "Inter", sans-serif; /* Applique la police Inter à tout le texte */
  -webkit-font-smoothing: antialiased; /* Rend le texte plus lisse sur les écrans */
  background-color: #111827; /* Couleur de fond gris très foncé */
  color: #f9fafb; /* Couleur du texte blanc cassé */
}
```

FICHIER **correction/src/pages/Home.jsx** 18 lignes

```jsx
import "../styles.css";
import Header from "../components/Header";
import ProductList from "../components/ProductList";
```

```jsx
import Cart from "../components/Cart";

function Home() {
  return (
    <main className="page">
      <Header />
      <div className="layout">
        <ProductList />
        <Cart />
      </div>
    </main>
  );
}

export default Home;
```

FICHIER **correction/src/components/Header.jsx** 18 lignes

```jsx
import { useSelector } from "react-redux";
import { selectCartCount } from "../store/cartSlice";

function Header() {
  // Le nombre d'articles vient du store : il se met à jour partout en même temps
  const count = useSelector(selectCartCount);

  return (
    <header className="header">
      <h1 className="header-title">Apple Store</h1>
      <div className="cart-badge">
        🛒 Panier <span className="cart-count">{count}</span>
      </div>
    </header>
  );
}

export default Header;
```

FICHIER **correction/src/components/ProductList.jsx** 17 lignes

```jsx
import products from "../data/products";
import ProductCard from "./ProductCard";

function ProductList() {
  return (
    <section>
      <h2 className="section-title">Nos produits</h2>
      <div className="products">
        {products.map((product) => (
          <ProductCard key={product.id} product={product} />
        ))}
      </div>
    </section>
  );
}

export default ProductList;
```

FICHIER **correction/src/components/Cart.jsx** 57 lignes

```jsx
import { useDispatch, useSelector } from "react-redux";
import {
  clearCart,
  decrement,
  increment,
  removeFromCart,
```

```
  selectCartItems,
  selectCartTotal,
} from "../store/cartSlice";
import { formatPrice } from "../utils";

function Cart() {
  const items = useSelector(selectCartItems);
  const total = useSelector(selectCartTotal);
  const dispatch = useDispatch();

  return (
    <aside className="card cart">
      <h2 className="section-title">Mon panier</h2>

      {items.length === 0 ? (
        <p className="empty">Votre panier est vide.</p>
      ) : (
        <>
          <ul className="cart-lines">
            {items.map((item) => (
              <li key={item.id} className="cart-line">
                <span className="cart-name">
                  {item.icon} {item.name}
                </span>
                <div className="qty">
                  <button onClick={() => dispatch(decrement(item.id))}>−</button>
                  <span>{item.quantity}</span>
                  <button onClick={() => dispatch(increment(item.id))}>+</button>
                </div>
                <span className="cart-price">{formatPrice(item.price * item.quantity)}</span>
                <button className="remove" onClick={() => dispatch(removeFromCart(item.id))}>
                  ×
                </button>
              </li>
            ))}
          </ul>

          <div className="row total">
            <span>Total</span>
            <span className="price">{formatPrice(total)}</span>
          </div>
          <button className="btn btn-secondary" onClick={() => dispatch(clearCart())}>
            Vider le panier
          </button>
        </>
      )}
    </aside>
  );
}

export default Cart;
```

FICHIER **correction/src/components/ProductCard.jsx** 31 lignes

```
import { useDispatch } from "react-redux";
import { addToCart } from "../store/cartSlice";
import { formatPrice } from "../utils";

function ProductCard({ product }) {
  const dispatch = useDispatch();

  return (
    <div className="card">
      <div className="product-icon">{product.icon}</div>
      <h3 className="product-name">{product.name}</h3>
```

```
      <div className="color">
        <span className="circle" style={{ backgroundColor: product.colorCode }}></span>
        <span>{product.color}</span>
      </div>
      <span className={`badge ${product.available ? "available" : "unavailable"}`}>
        {product.available ? "Available" : "Not Available"}
      </span>
      <p className="price">{formatPrice(product.price)}</p>
      <button
        className="btn"
        disabled={!product.available}
        onClick={() => dispatch(addToCart(product))}
      >
        Ajouter au panier
      </button>
    </div>
  );
}

export default ProductCard;
```

FICHIER `correction/src/store/store.js` — 9 lignes

```
import { configureStore } from "@reduxjs/toolkit";
import cartReducer from "./cartSlice";

export const store = configureStore({
  reducer: {
    cart: cartReducer,
  },
  devTools: true, // optionnel (par défaut true en dev)
});
```

FICHIER `correction/src/store/cartSlice.js` — 62 lignes

```
import { createSlice } from "@reduxjs/toolkit";

// Le panier : une liste de lignes { id, name, price, icon, quantity }
const initialState = {
  items: [],
};

const cartSlice = createSlice({
  name: "cart",
  initialState,
  reducers: {
    // payload = le produit à ajouter
    addToCart(state, action) {
      const product = action.payload;
      const item = state.items.find((i) => i.id === product.id);
      if (item) {
        item.quantity += 1; // déjà dans le panier : on augmente la quantité
      } else {
        state.items.push({
          id: product.id,
          name: product.name,
          price: product.price,
          icon: product.icon,
          quantity: 1,
        });
      }
    },
    // payload = l'id du produit
    increment(state, action) {
      const item = state.items.find((i) => i.id === action.payload);
```

```
      if (item) item.quantity += 1;
    },
    // payload = l'id du produit ; à 0, la ligne disparaît
    decrement(state, action) {
      const item = state.items.find((i) => i.id === action.payload);
      if (!item) return;
      item.quantity -= 1;
      if (item.quantity === 0) {
        state.items = state.items.filter((i) => i.id !== action.payload);
      }
    },
    // payload = l'id du produit
    removeFromCart(state, action) {
      state.items = state.items.filter((i) => i.id !== action.payload);
    },
    clearCart(state) {
      state.items = [];
    },
  },
});

export const { addToCart, increment, decrement, removeFromCart, clearCart } =
  cartSlice.actions;

// Selectors : lire le state du panier depuis n'importe quel composant
export const selectCartItems = (state) => state.cart.items;
export const selectCartCount = (state) =>
  state.cart.items.reduce((total, item) => total + item.quantity, 0);
export const selectCartTotal = (state) =>
  state.cart.items.reduce((total, item) => total + item.price * item.quantity, 0);

export default cartSlice.reducer;
```

FICHIER **correction/src/data/products.js** 11 lignes

```
// Les produits de la boutique (données en dur, comme un catalogue)
const products = [
  { id: 1, name: "MacBook Air 13\"", icon: "💻", color: "Minuit", colorCode: "#1f2a44", price: 1199, available: true },
  { id: 2, name: "iPhone 16", icon: "📱", color: "Bleu", colorCode: "#7aa6d6", price: 969, available: true },
  { id: 3, name: "AirPods Pro 2", icon: "🎧", color: "Blanc", colorCode: "#f5f5f7", price: 279, available: true },
  { id: 4, name: "Apple Watch Series 10", icon: "⌚", color: "Argent", colorCode: "#c0c4c8", price: 449, available: true },
  { id: 5, name: "iMac 24\"", icon: "🖥️", color: "Violet", colorCode: "#9b8bd6", price: 1499, available: false },
  { id: 6, name: "AirTag", icon: "📍", color: "Blanc", colorCode: "#f5f5f7", price: 35, available: true },
];

export default products;
```

FICHIER **correction/src/utils.js** 3 lignes

```
// Formate un prix en euros : 1199 => "1 199 €"
export const formatPrice = (price) =>
  price.toLocaleString("fr-FR", { style: "currency", currency: "EUR", maximumFractionDigits: 0 });
```

FICHIER **correction/index.html** 13 lignes

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
```

```
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Apple Store</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```